

Microservices Production Guide — The 8 Fallacies of Distributed Systems

A complete deep-dive study guide covering what goes wrong in production microservices systems and how to fix it, with real-world BFSI (Banking, Financial Services & Insurance) examples and Java code.

The Foundation — Monolith vs Microservices

In a monolith, all services run inside one JVM and communicate via direct method calls in memory — zero network overhead, zero serialization cost, zero failure due to network.

```
One Java Application (one JVM)
├─ UserService.java
├─ OrderService.java
├─ PaymentService.java
└─ NotificationService.java

// Direct method call – nanoseconds, no failure
paymentService.processPayment(order);
```

In microservices, every service call becomes a network hop:

```
OrderService → HTTP Request → Network → PaymentService
```

This one fact is the root cause of 80% of microservices production issues.

The journey of a single network request:

```
Service A
  → OS network stack
  → Router
  → Switch
  → Another Router
  → Service B's OS
  → Service B
```

Any single point in this chain can fail.

The 8 Fallacies of Distributed Systems

Identified in 1994 by engineers at Sun Microsystems, these are 8 wrong assumptions developers make about networks. Every microservices production incident traces back to one or more of these.

Fallacy 1 — "The Network is Reliable"

What Goes Wrong

When Service A calls Service B, three failure modes exist:

1. Request never reaches Service B
2. Request reaches Service B — but response never comes back (**most dangerous**)
3. Both request and response sent — but connection drops mid-transfer

Why #2 is most dangerous:

```
Service A sends "process payment of ₹500"
Request reaches Service B
Service B processes payment ✓
Service B sends response
Network drops HERE ← the problem
Service A never gets response
```

```
Now Service A has two choices:
  Retry → payment processed twice 🤖
  Don't retry → payment lost 🤖
Both options are wrong.
```

This is called the **Two Generals Problem** — a fundamental problem in distributed systems with no perfect solution.

Real production code that hides this problem:

```
// Service A
try {
    Response response = restTemplate.post("/payment", request);
    // Did payment happen? We have no idea if network dropped
} catch (Exception e) {
    // Was payment processed before exception?
    // We don't know!
}
```

The catch block tells you nothing about what happened on Service B.

Real Incident: Food delivery app — Order Service calls Payment Service → Payment charges ₹450 ✓ → Network drops → Order Service retries → Payment charges ₹450 again → Customer charged ₹900 for one order. This has happened at Swiggy, Zomato, Amazon.

Solution — Idempotency Keys

Step 1: Service A generates a unique key per request:

```
String idempotencyKey = UUID.randomUUID().toString();
headers.add("Idempotency-Key", idempotencyKey);
restTemplate.post("/payment", request, headers);
```

Step 2: Service B checks before processing:

```
public Response processPayment(Request request, String idempotencyKey) {
    if (idempotencyStore.exists(idempotencyKey)) {
        return idempotencyStore.get(idempotencyKey); // Return cached response
    }
    Response response = doPayment(request);
    idempotencyStore.save(idempotencyKey, response);
    return response;
}
```

Now even if Service A retries 10 times — payment happens exactly once.

Definition: An operation is idempotent if doing it multiple times gives the same result as doing it once.

Operation	Why Critical
Payment processing	Duplicate charge
Account debit	Double debit
Loan disbursement	Double disbursement
KYC verification	Duplicate records
OTP generation	Security issue

Solution — Circuit Breaker (prevents Cascading Failure)

CLOSED → (failures cross threshold) → OPEN → (wait 30s) → HALF-OPEN → (test) → CLOSED

- If Service B fails 5 times in a row → stop calling it for 30 seconds
- Give it time to recover
- After 30s → try one request → if success, resume

Tools: Resilience4j, Hystrix

Solution — Exponential Backoff with Jitter (prevents Retry Storm)

```
Retry 1: wait 1s
Retry 2: wait 2s
Retry 3: wait 4s
Retry 4: wait 8s
+ add random jitter so all clients don't retry at same millisecond
Max 3 retries then fail fast
```

Fallacy 2 — "Latency is Zero"

The Scale of the Problem

Call Type	Latency
Monolith method call	~100 nanoseconds
Microservice REST call	5ms–500ms
Difference	50,000x–5,000,000x slower

Sequential service chain:

```
Client Request
  → API Gateway      (10ms)
  → Auth Service     (20ms)
  → Account Service  (30ms)
    → Fraud Service (50ms) ← if this becomes 500ms due to DB load
      → Payment      (40ms)
        → Notify     (20ms)
```

Normal total = 170ms

When Fraud Service DB slows: 620ms per request

The Hidden Killer — P99 Latency

Most developers look at average latency. P99 is what matters in production.

```
P50 = 50ms    → 50% of requests are faster
P90 = 200ms   → 90% of requests are faster
P99 = 2000ms  → 99% of requests are faster
P999 = 8000ms → 99.9% of requests are faster
```

If you have 1 million requests/day — P99 of 2000ms means 10,000 users experience 2-second response time every single day.

Why P99 is high even when average looks fine:

```
98 requests: 50ms each
2 requests: 5000ms (hit slow DB query)

Average = (98×50 + 2×5000) / 100 = 149ms ← looks acceptable
P99 = 5000ms ← 1% of users wait 5 seconds
```

Root causes of high P99:

- JVM Garbage Collection pauses (200–500ms freezes)
- DB connection pool exhaustion (requests queue waiting for a connection)
- Thread pool exhaustion (Service A has 50 threads, all busy waiting for slow Service B)

Solutions

Solution 1 — Parallel Calls (CompletableFuture)

```
// BAD – Sequential (200ms total)
User user      = userService.getUser(id);           // 50ms
Account account = accountService.get(id);           // 50ms
List txns      = transactionService.get(id);        // 50ms
List notifs    = notificationService.get(id);       // 50ms

// GOOD – Parallel (50ms total)
CompletableFuture<User> userFuture =
    CompletableFuture.supplyAsync(() -> userService.getUser(id));
CompletableFuture<Account> accountFuture =
    CompletableFuture.supplyAsync(() -> accountService.get(id));
CompletableFuture<List> txnFuture =
    CompletableFuture.supplyAsync(() -> transactionService.get(id));

CompletableFuture.allOf(userFuture, accountFuture, txnFuture).join();
// 4 calls × 50ms sequential = 200ms → 4 calls × 50ms parallel = 50ms
```

Solution 2 — Caching

```
@Cacheable(value = "accounts", key = "#accountId")
public Account getAccount(String accountId) {
    return accountService.getAccount(accountId);
    // Hits network ONCE – subsequent calls return from cache in <1ms
}
```

Cache in BFSI ✓	Do NOT Cache ✗
User profile (changes rarely)	Account balance (must be real time)
Exchange rates (update every few minutes)	Transaction status (must be real time)
Product catalog	Fraud signals (must be real time)

Solution 3 — Aggressive Timeouts

```
// BAD – waits forever
restTemplate.get("http://fraud-service/check");

// GOOD – fail fast after 2 seconds
// Set connect timeout = 1s, read timeout = 2s
```

Rule of thumb for BFSI:

- Internal service calls → 2–3 second timeout
- External payment gateway → 5–10 second timeout
- Never more than 30 seconds for anything

Solution 4 — Async for Non-Critical Operations

```
// Payment – SYNCHRONOUS (user is waiting)
paymentService.processPayment(request);

// Notification – ASYNCHRONOUS (user doesn't wait)
kafkaTemplate.send("notifications", event);
// Return response to user immediately
// Notification service handles in background
```

Fallacy 3 — "Bandwidth is Infinite"

The Core Problem

In a monolith, data moves as object references in memory — zero bytes transferred.
In microservices, every call serializes data to JSON and sends bytes over the wire.

Over-fetching: Returning more data than the caller needs.

```
// Dashboard API only needs: accountId, balance, ownerName
// But Account Service returns:
{
  "accountId": "123",
  "balance": 1000.00,
  "ownerName": "John",
  "transactions": [...500 transactions...], ← NOT NEEDED
  "auditLogs": [...1000 entries...]          ← NOT NEEDED
}
// One response = 2MB
// 1000 requests/second × 2MB = 2GB/second bandwidth consumed
```

Under-fetching / N+1 Problem: Not enough data in one call, so you make N additional calls.

```
// BAD – N+1 problem in a loan management system
List<Loan> loans = loanService.getAllLoans(); // 1 call
for (Loan loan : loans) {
```

```
User borrower = userService.getUser(loan.getUserId()); // 1 call PER LOAN
loan.setBorrower(borrower);
}
// 100 loans = 101 network calls = 101 × 20ms = 2020ms just for this API
```

Solutions

Solution1 — Projections (return only what's needed)

```
// BAD – returns everything
public Account getAccount(String id) {
    return accountRepository.findById(id);
}

// GOOD – return only what caller needs
public AccountSummary getAccountSummary(String id) {
    return accountRepository.findSummaryById(id);
    // Only returns: accountId, balance, ownerName
}
```

Solution2 — Pagination

```
// BAD – returns all 10,000 transactions
List<Transaction> getAllTransactions(String accountId);

// GOOD – returns 20 at a time
Page<Transaction> getTransactions(String accountId, int page, int size);
```

Solution3 — Batch API (solves N+1)

```
// BAD – 100 separate calls
for (Loan loan : loans) {
    userService.getUser(loan.getUserId());
}

// GOOD – 1 call for all
List<String> userIds = loans.stream().map(Loan::getUserId).collect(toList());
Map<String, User> users = userService.getUsersByIds(userIds);
// 100 calls → 1 call
```

Solution4 — Response Compression

```
server:
  compression:
    enabled: true
    mime-types: application/json
    min-response-size: 1024
# JSON compresses 70-80% – 2MB → 400KB
```

Rules of thumb:

- No API should return more than 100KB
- Always paginate lists
- Never do N+1 calls

Fallacy 4 — "The Network is Secure"

The Threat Model

Most developers only think about external attackers. Real production breaches come from:

Threat	Description
External attacker	Someone outside your network
Internal attacker (Zero Trust)	A compromised service inside your own network
Man in the Middle	Someone intercepting traffic between two services
Replay Attack	Attacker captures a valid request and sends it again

Real BFSI Incident: 2016 Bangladesh Bank Heist — \$81 million stolen. Attackers got inside the network, impersonated internal services (SWIFT messages), and transferred funds because internal services trusted each other blindly with no service-to-service authentication.

Problem 1 — Plain HTTP Between Services

```
# Dangerous – plain text inside your own network
uri: http://account-service:8082
# Attacker inside network sees: account numbers, balances, JWT tokens
```

Solution: mTLS (Mutual TLS)

	TLS	mTLS
Client verifies Server	✓	✓
Server verifies Client	✗	✓

Analogy: TLS = you show your ID at the door. mTLS = both you AND the security guard show IDs to each other.

Problem 2 — No Service-to-Service Authentication

Any service can call any other service — including a compromised one.

Solution: Service Identity (JWT per service or API Keys per service)

```
// Service A calls Service B – includes its own identity token
headers.add("X-Service-Token", accountServiceJWT);
```



```
// Service B validates:  
// 1. Is this a valid token?  
// 2. Is this from account-service specifically?  
// 3. Is account-service allowed to call me?
```

Problem 3 — Replay Attack

Step 1: User sends legitimate payment request
POST /payment {"amount": 1000, "to": "ACC123"}
Authorization: Bearer eyJhbG...

Step 2: Attacker captures this exact request

Step 3: Attacker replays same request 10 times

Step 4: ₹10,000 transferred instead of ₹1,000

Solution: Request Signing + Timestamp + Nonce

```
// Request includes:  
X-Timestamp: 1704067200      // when request was made  
X-Nonce: a3f8b2c9           // random number used exactly once  
X-Signature: HMAC(body + timestamp + nonce, secretKey)  
  
// Server validates:  
if (request.timestamp < now - 5minutes) throw new ReplayAttackException();  
if (nonceStore.exists(request.nonce)) throw new ReplayAttackException();  
if (!hmac.verify(request)) throw new InvalidSignatureException();  
nonceStore.save(request.nonce, TTL = 5minutes);
```

Problem 4 — Sensitive Data in Logs

```
// DANGEROUS – violates PCI-DSS  
log.info("Processing payment: {}", paymentRequest.toString());  
// Logs: cardNumber=4532015112830366, cvv=123, accountNumber=1234567890
```

Solution: Data Masking

```
log.info("Processing payment for account: {}", maskAccount(payment.getAccountNumber()));  
// Logs: Processing payment for account: ****7890
```

Fields that must NEVER appear in BFSI logs:

- Card numbers, CVV
- Account numbers, Passwords, OTPs
- Aadhaar numbers, PAN numbers

Problem 5 — Secrets in Code

```
# NEVER do this – if pushed to GitHub, everyone sees your prod secrets
db.password=mypassword123
jwt.secret=supersecretkey
```

Solution: Environment Variables + Secret Managers

```
spring:
  datasource:
    password: ${DB_PASSWORD}    # from environment
  jwt:
    secret: ${JWT_SECRET}      # from environment
```

Production secret managers: AWS Secrets Manager, HashiCorp Vault, Kubernetes Secrets

Zero Trust Principle

Never trust, always verify.

Even if a request comes from inside your own network — verify identity, verify permissions, verify integrity.

Fallacy 5 — "The Topology Doesn't Change"

Why Hardcoded IPs Break in Modern Infrastructure

```
// Old way – works for physical servers
restTemplate.get("http://192.168.1.10:8082/accounts");
```

Reason 1 — Cloud instances have dynamic IPs:

```
EC2 instance starts → IP: 54.23.187.91
Instance crashes, AWS restarts → IP: 54.23.191.45 ← completely different!
```

Reason 2 — Containers change IPs constantly:

```
Docker assigns: 172.18.0.5
Container crashes, Docker restarts: 172.18.0.8 ← different!
```

Reason 3 — Autoscaling:

```
9:00 AM → 2 instances of Payment Service
10:00 AM → Traffic spikes → 10 instances (10 different IPs)
11:00 AM → Traffic drops → back to 2
```

Solution — Service Discovery

Three components:

1. **Service Registry** — central directory of all running services
2. **Registration** — each service announces itself on startup
3. **Discovery** — services look up others by name, not IP

How Eureka works:

```
Step 1: Account Service starts → registers with Eureka
       "I am account-service, running at 172.18.0.5:8082, I am healthy"

Step 2: Registry stores:
       account-service → 172.18.0.5:8082 ✓

Step 3: Other services discover by name (no hardcoded IP):
       restTemplate.get("http://account-service/accounts/" + id);
       // Eureka resolves name → actual IP automatically

Step 4: Service crashes → Eureka removes it after heartbeat timeout
Step 5: Service restarts with new IP → re-registers automatically
```

Eureka vs Kubernetes Service Discovery

Feature	Eureka	Kubernetes
Registration	App registers itself	K8s knows automatically (it started the container)
Health check	App sends heartbeat	K8s actively probes <code>/actuator/health</code> every 10s
Stale entries	Yes — 30–90s delay	No — immediate detection
Load balancing	Client side (your code)	Built-in (kube-proxy)
Single point of failure	Yes (unless clustered)	No (etcd is distributed, 3–5 nodes)
App alive but unhealthy	Undetected — shows healthy	Detected — K8s restarts pod

Key difference in philosophy:

- **Eureka** = asking employees to sign in themselves
- **Kubernetes** = HR already knows who works here because they hired them

```
# Kubernetes health check – infrastructure level
livenessProbe:
  httpGet:
    path: /actuator/health
    port: 8082
  periodSeconds: 10
  failureThreshold: 3
# Fails 3 times → Kubernetes kills and restarts pod automatically
```

When to use what:

- Learning / Local development → Eureka (simple setup, good for understanding)
- Production at scale → Kubernetes Service Discovery (what Razorpay, PhonePe, Zepto use)

Fallacy 6 — "There is Only One Administrator"

The Multiplication Problem

In a monolith: one team, one config file, one deployment, one person knows everything.

In microservices with 6 services:

- 6 teams (or sub-teams)
- 6 codebases, 6 deployment pipelines
- 6 config files, 6 databases
- No single person controls everything

Problem 1 — Configuration Drift

Config files slowly become inconsistent across services:

```
account-service:      timeout: 30s,  retry: 3, log-level: INFO
transaction-service: timeout: 10s,  retry: 5, log-level: DEBUG
notification-service: timeout: 60s, retry: 1, log-level: INFO
```

When debugging: "Which service has which timeout?" — nobody knows.

Problem 2 — Config Change Causes Outage

```
# Developer changes DB pool from 10 → 2
spring.datasource.maximum-pool-size: 2
```

Result: Peak traffic hits → only 2 DB connections → 998 requests queue → 30-second response times → production outage from one number change. Takes hours to find.

Problem 3 — Secret Management Chaos

DB password expires. Need to update in:

- account-service ✓
- auth-service ✓
- transaction-service ← forgot → transaction-service goes down

Solution 1 — Centralized Configuration (Spring Cloud Config)

```
Config Server
├─ Git Repository
│   ├── account-service.yml
│   ├── auth-service.yml
│   ├── transaction-service.yml
│   └─ application.yml (shared config for all)
```

```
# Each service just points to config server
spring:
  config:
    import: configserver:http://config-server:8888
```

Benefits: One place to change → all services get updated → config versioned in Git → full audit trail → rollback config like code.

Solution 2 — Environment-Specific Config

```
Config Server / Git
├─ account-service-dev.yml      (log-level: DEBUG, timeout: 60s)
├─ account-service-staging.yml
└─ account-service-prod.yml     (log-level: ERROR, timeout: 5s)
```

Solution 3 — Centralized Secret Management

```
HashiCorp Vault / AWS Secrets Manager
├─ /banking/prod/db-password
├─ /banking/prod/jwt-secret
├─ /banking/prod/kafka-api-key
└─ /banking/prod/redis-password
```

Benefits: Secrets in one place, automatic rotation, audit log of who accessed what, update once → all services get new value.

Solution 4 — Service Ownership + Runbooks

Every service must define:

- Owner (team name)
- On-call rotation schedule
- Runbook — step-by-step guide for every known issue

Example runbook entry:

```
Problem: Account service high latency
Step 1: Check /actuator/metrics for DB pool exhaustion
```

```
Step 2: Check Grafana dashboard
Step 3: Check Kafka consumer lag
Step 4: If DB pool exhausted → restart with higher pool size
Step 5: Escalate to DBA if still failing
```

How Razorpay manages config: Central Git repo → PR review → staging deployment → canary deployment (5% traffic first, then 100%) → automated rollback if error rate spikes → full audit trail.

Fallacy 7 — "Transport Cost is Zero"

What is Transport Cost?

Every microservice call pays two costs:

1. **Serialization** — Java object → JSON bytes (to send)
2. **Deserialization** — bytes → Java object (on receive)

In a monolith: passes object reference in memory — zero cost.

In microservices: every call converts every object, every time.

What Happens During JSON Serialization

Send side:

- ```
Step 1: Read every field via reflection
Step 2: Convert each field to string
Step 3: Add JSON syntax { " " : " " }
Step 4: Convert string to bytes
Step 5: Send bytes
```

Receive side:

- ```
Step 1: Receive bytes
Step 2: Convert bytes to string
Step 3: Parse JSON character by character
Step 4: Match key to Java field via reflection
Step 5: Set each field value
```

Reflection is the expensive part — Java inspects its own code at runtime. It is 10–100x slower than direct field access.

At scale:

10,000 requests/second → 20,000 reflection operations/second

CPU Breakdown on a Transaction Service:

```
Business logic (fraud check, DB query): 30%
JSON serialization/deserialization:    35% ← more than business logic!
Network I/O:                           5%
```

35% of CPU doing nothing useful — just converting objects.

The JSON Problem — Text-Based

JSON is human-readable but machines don't need to read it. Every message carries field names repeated:

```
{"accountId": "ACC123", "balance": 1000.00, "ownerName": "John Doe"}
```

The field name "accountId" is sent with every single message. The machine has to parse text character by character to find values.

The Solution — Protocol Buffers (Protobuf)

Define your schema in a .proto file:

```
message Account {  
  string account_id = 1;  
  double balance    = 2;  
  string owner_name = 3;  
}
```

Instead of sending field name "accountId" as text — it sends the number 1. The receiver already knows: field 1 = accountId.

Size comparison for the same data:

Format	Size	Notes
JSON	185 bytes	Human readable, field names repeated
Protobuf	52 bytes	Binary, field names are numbers
Difference	3.5x smaller	No names, no quotes, no braces

Performance comparison:

	JSON	Protobuf
Payload size	185 bytes	52 bytes
Serialization speed	Baseline	~5x faster
CPU usage	High (reflection)	Low (direct binary)
Human readable	Yes ✓	No ✗

Real world result: Netflix switched internal service communication from JSON to Protobuf → 60% reduction in bandwidth, 30% reduction in CPU on serialization.

When to Use What

Use Case	Format
External APIs (mobile app, third party)	JSON/REST — human readable, easy to consume
Internal service-to-service	Protobuf + gRPC — performance matters

Fallacy 8 — "The Network is Homogeneous"

The Assumption vs Reality

Assumption: All services use Java, REST, JSON, same library versions, same data formats.

Reality of a production BFSI system:

```
auth-service           → Java, Spring Boot
payment-service        → Java, Spring Boot
fraud-service          → Python, FastAPI (ML model)
notification-service   → Node.js
mobile-app             → Swift / Kotlin
third-party bank API   → COBOL (yes, still exists in banking)
```

Problem 1 — Date Format Mismatch

```
// Java service sends:
{"transactionDate": "2024-01-15T10:30:00.000+05:30"}
```

```
// Python service expects:
{"transactionDate": "2024-01-15"}
```

```
// Python service receives Java format → crash
```

Problem 2 — Null Handling Differences

```
// Node.js receives: {"middleName": null}
user.middleName.toUpperCase()
// Cannot read property 'toUpperCase' of null → CRASH
```

Problem 3 — Number Precision (Dangerous in BFSI)

```
// Java sends large balance
{"balance": 999999999999.99}
```

```
// JavaScript receives and silently loses precision:
console.log(999999999999.99)
// Output: 10000000000000 ← WRONG – no error, no crash, just wrong data
```


Problem 4 — Breaking API Changes (Most Common Production Incident)

```
// Team A renames a field (thinks it's a cleanup)
// Old: {"accountId": "ACC123"}
// New: {"id": "ACC123"} ← RENAMED

// Team A deploys.
// Transaction-service was reading "accountId" → now gets null
// NullPointerException thrown – Team B has no idea why
// Team B: "We didn't change anything!"
// Team A: "We just renamed a field, no big deal"
// Production is down. Finger pointing begins.
```

Problem 5 — Version Mismatch During Rolling Deployment

```
During deployment of v2:
  Instance 1: v1 → response has NO "availableBalance" field
  Instance 2: v2 → response HAS "availableBalance" field

Same service, two different response shapes
Consumer reads "availableBalance" → sometimes null, sometimes has value
Intermittent bugs – hardest to debug
```

Solution 1 — API Contracts (OpenAPI / Swagger)

Define the contract first — both sides agree on field names, types, date formats, nullable fields.

```
paths:
  /accounts/{id}:
    get:
      responses:
        200:
          schema:
            properties:
              accountId:
                type: string
              balance:
                type: number
                format: double
              transactionDate:
                type: string
                format: date-time ← exact format defined
```

Solution 2 — Consumer-Driven Contract Testing

Transaction-service (consumer) writes a test:

```
"I expect account-service to return: accountId as string, balance as double, NOT null  
accountId"
```

This test runs against account-service **on every deployment**.

If Team A renames `accountId` → `id` → **contract test FAILS** → **deployment BLOCKED** → Team A notified before reaching production.

Tool: Pact — used by Atlassian, ING Bank, many BFSI companies.

Solution 3 — Backward Compatible Changes Only

```
// ✓ SAFE — adding a new field (old consumers ignore it)  
{  
  "accountId": "ACC123",           ← kept  
  "balance": 1000.00,             ← kept  
  "availableBalance": 950.00      ← NEW — safe to add  
}  
  
// ✗ NEVER DO — renaming breaks all consumers  
{  
  "id": "ACC123"                  ← renamed from accountId — every consumer breaks  
}
```

Solution 4 — API Versioning

```
/v1/accounts/123 → old contract, kept alive  
/v2/accounts/123 → new contract  
  
Consumers migrate to v2 at their own pace.  
v1 deprecated after 3-6 months with advance notice.
```

Master Summary — All 8 Fallacies

#	Fallacy	Real Production Problem	Solution
1	Network is reliable	Requests lost or duplicated (duplicate payments)	Idempotency Keys + Circuit Breaker + Outbox Pattern
2	Latency is zero	Slow service chains, high P99, cascading slowness	Parallel calls (CompletableFuture) + Caching + Timeouts
3	Bandwidth is infinite	Large payloads, N+1 problem, bandwidth saturation	Projections + Pagination + Batch APIs + Compression
4	Network is secure	Data interception, replay attacks, secrets leaked	mTLS + Request Signing (HMAC+Nonce) + Vault

#	Fallacy	Real Production Problem	Solution
5	Topology doesn't change	Hardcoded IPs break when containers restart or scale	Service Discovery (Eureka / Kubernetes)
6	One administrator	Config drift, secret chaos, blame-driven debugging	Spring Cloud Config + Vault + Runbooks
7	Transport cost is zero	High CPU on serialization, large JSON payloads	Protobuf + gRPC for internal services
8	Network is homogeneous	Silent data corruption, breaking API changes	API Contracts (OpenAPI) + Contract Testing (Pact)

Key Patterns Quick Reference

Problem	Pattern	Tool
Duplicate payment on retry	Idempotency Keys	UUID + DB/Redis check
One slow service kills all	Circuit Breaker	Resilience4j
Retry storm on failure	Exponential Backoff with Jitter	Resilience4j
Kafka duplicate processing	Idempotent Consumer	Custom + DB
Kafka poison pill message	Dead Letter Queue (DLQ)	Kafka native
Message lost if Kafka down	Outbox Pattern	DB transactional outbox
Kafka out-of-order events	Partition Key	Account ID as key
Sequential API calls slow	Parallel Calls	CompletableFuture
Repeated expensive calls	Caching	Redis + @Cacheable
N+1 network calls	Batch API	Custom batch endpoint
Service IP keeps changing	Service Discovery	Eureka / Kubernetes
Config inconsistent	Central Config	Spring Cloud Config
Secrets in code/git	Secret Manager	HashiCorp Vault / AWS SM
JSON overhead at scale	Binary serialization	Protobuf + gRPC
Silent breaking API change	Contract Testing	Pact

What's Next — Topics Not Yet Covered

The following topics are planned for study in subsequent sessions:

1. Data Patterns & Failures

- Why you can't use a shared database in microservices
- Distributed transactions — the hardest problem
- Saga Pattern (choreography vs orchestration)

- CQRS — separate read/write scaling
- Event Sourcing — complete audit trail for banking
- 2. Resilience Patterns — Bulkhead, Rate Limiting, Fallback
- 3. Security Patterns — OAuth2, JWT flows, API Gateway auth
- 4. Observability Patterns — Distributed tracing, structured logging, metrics (Prometheus + Grafana)
- 5. Deployment Patterns — Blue-Green, Canary, Feature Flags

Study session conducted: May 8, 2026 / Focus: 8 Fallacies of Distributed Systems with BFSI production examples